

27 Marzo 2026

Actividad 2

Equipo 5:

Diego Couto

Fernando Samaniego

Santiago Atahualpa

1. Configuración del entorno

1. Inicia sesión en AWS Learner Lab y asegúrate de que el entorno está activo.
2. Accede a la consola de AWS y verifica que la región seleccionada es us-east-1 o us-west-2, conforme a las restricciones del laboratorio.
3. Usa AWS CloudShell o una instancia de EC2 con acceso SSH para ejecutar los scripts.

2. Automatización con Python y Boto3

1. Instala y configura Boto3 en el entorno si no está disponible:

```
pip install boto3
```

Ejecución:

```
~ $ pip show boto3
Name: boto3
Version: 1.42.1
Summary: The AWS SDK for Python
Home-page: https://github.com/boto/boto3
Author: Amazon Web Services
Author-email:
License: Apache-2.0
Location: /usr/local/lib/python3.9/site-packages
Requires: botocore, jmespath, s3transfer
Required-by: aws-sam-cli, aws-sam-translator, cassandra-sigv4, cqlsh-expansion, graphsh
~ $
```

2. Configura las credenciales de AWS si es necesario:

```
aws configure
```

Ejecución:

```
~ $ aws configure
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [us-east-1]:
```

```
Default output format [json]:
```

```
~ $
```

3. Desarrolla un script en Python (gestionar_ec2.py) que realice las siguientes acciones:

- Listar todas las instancias en la región y su estado actual.
- Iniciar una instancia específica si está detenida.
- Detener una instancia específica si está en ejecución.

4. Código base sugerido en Python:

```
import boto3

ec2 = boto3.client('ec2', region_name='us-east-1')

def listar_instancias():
    response = ec2.describe_instances()
    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            print(f"ID: {instance['InstanceId']}, Estado: {instance['State']['Name']}")

def gestionar_instancia(instancia_id, accion):
    if accion == "iniciar":
        ec2.start_instances(InstanceIds=[instancia_id])
        print(f"Instancia {instancia_id} iniciada.")
    elif accion == "detener":
        ec2.stop_instances(InstanceIds=[instancia_id])
        print(f"Instancia {instancia_id} detenida.")
```

```
if __name__ == "__main__":
    listar_instancias()
    gestionar_instancia("ID_INSTANCIA", "iniciar") # Reemplazar ID_INSTANCIA según corresponda.
```

Código final:

```
import boto3
import sys

# Configuración del cliente
ec2 = boto3.client('ec2', region_name='us-east-1')

def listar_instancias():
    print("--- Listado de Instancias ---")
    response = ec2.describe_instances()
    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            print(f"ID: {instance['InstanceId']} | Estado: {instance['State']['Name']}")
    print("-----\n")

def gestionar_instancia(instancia_id, accion):
    try:
        if accion == "iniciar":
            ec2.start_instances(InstanceIds=[instancia_id])
            print(f"Comando enviado: Iniciando {instancia_id}...")
        elif accion == "detener":
            ec2.stop_instances(InstanceIds=[instancia_id])
```

```

        print(f"Comando enviado: Deteniendo {instancia_id}...")
    else:
        print("Acción no válida. Usa 'iniciar' o 'detener'.")
except Exception as e:
    print(f"Error al procesar la instancia: {e}")

if __name__ == "__main__":
    # Si se pasan argumentos: python gestionar_ec2.py <id> <accion>
    if len(sys.argv) > 2:
        id_instancia = sys.argv[1]
        accion_usuario = sys.argv[2].lower()
        gestionar_instancia(id_instancia, accion_usuario)
    else:
        # Si no hay argumentos, solo lista
        listar_instancias()
        print("Uso: python gestionar_ec2.py <ID_INSTANCIA> <iniciar|detener>")

```

5. Automatización con Bash:

```

#!/bin/bash

# ID de la instancia que quieres gestionar
INSTANCIA_ID="i-0123456789abcdef0" # <--- REEMPLAZA CON TU ID REAL

# Obtener el día de la semana actual (1-7, donde 6 y 7 son fin de semana)
DIA_SEMANA=$(date +%u)

echo "=====

```

```

echo "Iniciando mantenimiento de EC2: $(date)"
echo "=====

# 1. Listar el estado actual de todas las instancias
echo "Estado actual de la región:"
python3 gestionar_ec2.py

# 2. Lógica de automatización por fin de semana
if [ "$DIA_SEMANA" -eq 6 ] || [ "$DIA_SEMANA" -eq 7 ]; then
    echo "Detección: Es fin de semana. Procediendo a detener la instancia por ahorro."
    python3 gestionar_ec2.py "$INSTANCIA_ID" detener
else
    echo "Detección: Es día laboral. No se realizarán paradas automáticas."
    # Opcional: Podrías poner aquí un comando para iniciarla si quieres que siempre esté prendida
    # python3 gestionar_ec2.py "$INSTANCIA_ID" iniciar
fi

echo "=====
echo "Proceso finalizado con éxito."

```

3. Crea un script en Bash (backup_s3.sh) para generar un respaldo de archivos y enviarlo a un bucket S3

1. El script debe:

- Comprimir un directorio específico.
- Subir el archivo comprimido a un bucket S3 en la misma región.

- Generar un log con detalles de la operación.

2. Código base sugerido en Bash:

```
#!/bin/bash

BUCKET_NAME="mi-bucket-ejemplo"
BACKUP_FILE="backup_$(date +%F).tar.gz"
LOG_FILE="backup.log"

echo "Iniciando respaldo..." >> $LOG_FILE
tar -czf $BACKUP_FILE /ruta/a/respaldo >> $LOG_FILE 2>&1

if aws s3 cp $BACKUP_FILE s3://$BUCKET_NAME/ >> $LOG_FILE 2>&1; then
    echo "Respaldo subido exitosamente." >> $LOG_FILE
else
    echo "Error en la subida del respaldo." >> $LOG_FILE
fi
```

Código Final:

```
#!/bin/bash

# --- Configuration ---
BUCKET_NAME="act-2-backup"
SOURCE_DIR="/home/cloudshell-user/to-backup" # Change this to the folder you want to back up
BACKUP_NAME="backup_$(date +%Y-%m-%d_%H%M%S).tar.gz"
```

```
LOG_FILE="backup.log"

# --- Execution ---
echo "--- Respaldo iniciado: $(date) ---" >> "$LOG_FILE"

# 1. Check if source directory exists
if [ ! -d "$SOURCE_DIR" ]; then
    echo "ERROR: El directorio $SOURCE_DIR no existe." >> "$LOG_FILE"
    exit 1
fi

# 2. Create the compressed archive
# 'c' create, 'z' gzip, 'f' file
tar -czf "$BACKUP_NAME" "$SOURCE_DIR" >> "$LOG_FILE" 2>&1

# 3. Upload to S3 and check for success
if aws s3 cp "$BACKUP_NAME" "s3://$BUCKET_NAME/" >> "$LOG_FILE" 2>&1; then
    echo "SUCCESS: Respaldo $BACKUP_NAME subido correctamente." >> "$LOG_FILE"
    # Optional: Remove local backup file after successful upload to save space
    # rm "$BACKUP_NAME"
else
    echo "ERROR: Falló la subida a S3. Revisa los permisos o la conexión." >> "$LOG_FILE"
fi

echo "--- Fin del proceso: $(date) ---" >> "$LOG_FILE"
echo "" >> "$LOG_FILE"
```

4. Ejecución y validación

1. Ejecuta el script de Python y verifica que las instancias se listan y gestionan correctamente.

Las instancias se listan correctamente:

```
code $ python3 gestionar_ec2.py
--- Listado de Instancias ---
ID: i-00e2ef44de3d9da57 | Estado: stopped
-----
Uso: python gestionar_ec2.py <ID_INSTANCIA> <iniciar|detener>
```

Ahora con los parámetros para iniciar la instancia listada:

```
code $ python3 gestionar_ec2.py i-00e2ef44de3d9da57 iniciar
Comando enviado: Iniciando i-00e2ef44de3d9da57...
```

La instancia inicia correctamente:

Instances (1) [Info](#)

Saved filter sets

Choose filter set ▼

☐	Name 	Instance ID	Instance state 
☐	devOps_debian	i-00e2ef44de3d9da57	 Running  



2. Ejecuta el script de Bash y comprueba que el archivo se genera y sube a S3 exitosamente.

```
code $ bash backup_s3.sh
code $ ls
auto_ec2.sh  backup_2026-03-28_185257.tar.gz  backup.log  backup_s3.sh  gestionar_ec2.py
```

act-2-backup Info

Objects | Metadata | Properties | Permissions | Metrics

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 in](#)

☐	Name	Type
☐	 backup_2026-03-28_185257.tar.gz	gz

3. Consulta los logs generados para asegurar que no hay errores en la ejecución.

```
code $ cat backup.log
--- Respaldo iniciado: Sat Mar 28 06:52:57 PM UTC 2026 ---
tar: Removing leading `/' from member names
upload: ./backup_2026-03-28_185257.tar.gz to s3://act-2-backup/backup_2026-03-28_185257.tar.gz
```

SUCCESS: Respaldo backup_2026-03-28_185257.tar.gz subido correctamente.

--- Fin del proceso: Sat Mar 28 06:52:58 PM UTC 2026 ---

code \$